

Reutilización basada en plantillas y composición como alternativa a librerías y OOP para la reutilización de código

Fernando Enrique Araoz Morales
Área de Digitales - ACIDE
Arequipa, Perú
fernando@araoz.dev

Abstract—Una de las metas más comunes para las empresas de desarrollo de software es la reutilización. Sin embargo, la experiencia empírica nos dice que si no se hace con cuidado, puede originar más problemas de los que soluciona. En este artículo se exploran las deficiencias de la propuesta de reutilización existente, y se presenta una nueva propuesta basada en paradigmas opuestos. Se encontró que este nuevo paradigma sería capaz de mitigar muchas de las deficiencias existentes, sirviendo como una base para el crecimiento de la empresa.

Index Terms—reusabilidad, componentes, librerías, plantillas.

I. INTRODUCCIÓN

En el área de digitales se lleva hablando de reusabilidad de código desde hace bastante tiempo. Sin embargo, no se ha logrado un consenso entre tanto la taxonomía del mismo, la implementación, el alcance, entre otros factores. Este documento propone un marco de trabajo completo para alcanzar la reutilización no solo de código, sino de los sistemas que desarrollamos.

El documento se divide de la siguiente manera: Primero se realiza un análisis de la propuesta existente, posteriormente se discute brevemente la nomenclatura de Taller vs. Fábrica. Se continúa detallando una nueva propuesta para la reutilización, y se concluye con limitaciones, recomendaciones y conclusiones. Como resultado, se encontró que este enfoque mitiga la rigidez de las librerías tradicionales, potenciando la agilidad y la alineación con el modelo de negocio.

II. CRÍTICA A LA PROPUESTA EXISTENTE

Se presentó una propuesta para la reutilización de sistemas recientemente, con muchísimas deficiencias y una reacción adversa en el equipo de desarrollo. Esta reacción proviene de los siguientes factores:

- **Falta de claridad:** Se presentó una lista de ideas y conceptos, sin especificar los detalles de la implementación. No se tienen claros los actores involucrados, el alcance de los cambios, la gobernanza, la resolución de conflictos, cómo las librerías interactuarán con los sistemas nuevos, cómo los sistemas nuevos interactuarán con las librerías, qué estilo de librerías se utilizará, quiénes son dueños del código, qué hacer en caso de que la librería entre en

conflicto con el modelo de negocio de una empresa. El no contar con respuestas claras y concretas abre paso a la especulación, y a que un número pequeño de actores imponga sus ideas por sobre las del equipo.

- **Falta de consenso:** Un número pequeño del equipo de trabajo ha presentado la idea, de este modo imponiendo sus ideas sobre todo el equipo. Esto llevará indudablemente a conflictos internos, y reduce la visión que pueden tener otros miembros, evitando el intercambio de ideas y que nuevas y mejores soluciones emerjan.
- **Falta de credibilidad:** En el proyecto de Colver ya se ha materializado una implementación de la propuesta de librerías. Inmediatamente derivó en conflictos entre el equipo, se presentaron preguntas y críticas que no fueron resueltas, y causaron fricción en la parte técnica. La pequeña sección que utilizó las librerías resultó de peor calidad que una implementación independiente, y se retrasó al equipo de trabajo. Volver a presentar la misma idea nebulosa a todo el equipo no resuelve los problemas presentados anteriormente.

El principal problema de la propuesta existente es que trata a los módulos como "librerías", que existen fuera del proyecto y por lo tanto son estáticas, no se pueden cambiar o adaptar para necesidades específicas. A continuación se presentan los problemas y limitaciones ordenados de más general a más específico, y productos de la rigidez mencionada.

A. Overengineering

La propuesta de librerías es demasiado extensa y no es adecuada para los sistemas que desarrollamos en la empresa. Se intentó replicar completamente toda la estructura utilizada por Odoo, es decir, muchísima funcionalidad que es útil en el contexto de un ERP completo y modular, pero no para un colegio privado pequeño [1]. Por ejemplo, a pesar de que Colver solo tiene una única sede, se planteó una estructura para múltiples sedes, con múltiples tipos de personal, con múltiples operaciones, etc. Esto es sin lugar a dudas, una solución para un problema que no existe, y que tuvo un impacto real en el proyecto.

Otro resultado de este paradigma es que en vez de pensar en una solución para Colver, se pensó primero en una solución

generalizada, aplicable a cualquier empresa de cualquier rubro. Si bien pensar en tal solución es bueno, el problema resulta en que ignora completamente el problema frente a nosotros. Colver pasó a segundo plano, e inevitablemente se encontraron lugares donde la lógica de negocio de Colver iba en contra de la lógica generalizada.

El Overengineering conlleva a los siguientes problemas, presentados por Jayashetty et al [1]:

- **Alineación del negocio:** En búsqueda de una solución reutilizable vamos en contra de las necesidades del cliente concreto.
- **Agilidad:** Un sistema generalizado reduce la agilidad del equipo, ya que tienen que trabajar en 2 lugares, "dar vueltas" para adecuarse a las limitaciones de la abstracción, o duplicar esfuerzos.
- **Redundancia:** Cuando la abstracción no encaja con el problema concreto, una de las soluciones más fáciles es ignorar la abstracción, y hacerlo de nuevo. Esto se manifestó claramente en Colver, la librería de `Persona` define `Nombre` y `Apellido`. Pero el `Estudiante` de Colver necesita `apellido paterno` y `materno`, separados. La solución fue ignorar la librería, y duplicar el código de manejo de nombres. Y esto volvería a pasar más adelante en el sistema de nuevo, con `Profesores`, `Administrativos`, `Auxiliar`, `Psicólogo`, etc.
- **Complejidad:** La generalización puede llevar fácilmente a complejidad. Para alcanzar una solución reutilizable por muchos negocios se debe pensar en todos los posibles casos y excepciones. Todo esto resulta en código complejo, para manejar múltiples usuarios. Wagner et al [2] muestran que a medida que el código se hace más abstracto, la complejidad tiende a aumentar. Una buena librería puede reducir la complejidad downstream [3], siempre que no se necesite modificar su comportamiento interno. Pero cuando surge esa necesidad, toda la complejidad se revela. Alcanzar tales abstracciones es importante, pero es una tarea sumamente difícil [4].
- **Costos operativos:** Desarrollar y mantener un sistema grande y complejo puede incrementar el costo operacional. Si las abstracciones no son las correctas el doble trabajo de mantenerlas incurre en más tiempo de los empleados. Nuevos trabajadores necesitan más tiempo para entender un sistema complejo, y si no son capaces de hacerlo representan un impedimento para el crecimiento del área [4].
- **Análisis por Parálisis:** Intentar desarrollar abstracciones generales **durante** el desarrollo de un proyecto puede llevar a una parálisis. Al pensar no solo en cómo abstraer para Colver, sino pensar en cómo abstraer para empresas futuras, corremos el riesgo de no hacer nada hasta obtener un diseño perfecto. Sería óptimo desarrollar abstracciones a medida que son necesarias, basadas en la experiencia concreta del equipo, y no en posibles futuras necesidades.

B. Ruptura de las prácticas de software

Existe una discusión acerca de si los patrones deberían cumplirse al 100%, o hasta qué nivel podemos ser pragmáticos y faltar a los ideales en lugar del proyecto. Es mi opinión que deberíamos seguir los patrones lo más fielmente posible, excepto en aquellos casos donde la agilidad de romperlos otorga agilidad y velocidad significativa.

En la implementación concreta observada se pierden los principios de DDD y OOP.

- **Diseño dirigido por el Dominio:** Al trabajar en librerías rígidas, no operamos sobre un Dominio en concreto, sino uno abstracto y general, y cuando surge una diferencia el modelo de DDD se pierde.
- **Dominio anémico:** Al trabajar en librerías genéricas, no se puede tener realmente un Dominio encapsulado. Es decir, la `Persona` no puede definir sus propias operaciones específicas para el dominio. En su lugar, `Persona` tiene una operación `Update`, que permite cualquier cambio. Es en ese momento que se pierde el DDD, y volvemos a objetos planos.
- **Abuso de herencia:** Una posible solución al problema de dominio anémico es utilizar la herencia de clases. Sin embargo, esta es una falsa solución y suele traer muchos más problemas de los que resuelve. La industria ahora aconseja evitar herencia y usar composición [5]–[11]. Más aún, al pensar en cómo manejar herencia en la base de datos se incurre en problemas: a) Tablas duplicadas en la base de datos, si cada clase hija crea su propia tabla, b) Múltiples tablas pequeñas, si cada superclase es su propia tabla, hay que buscar y ejecutar JOINS para cada superclase.
- **Principio de Sustitución de Liskov:** En el caso de `Persona` y `Estudiante`, `estudiante` reemplazó la lógica de nombres de `Persona`, al crear `apellido paterno/materno` e ignorar el único campo `Apellido` del padre. Al hacer esto se perdió el principio de sustitución de Liskov. Una `Persona` ya no se puede usar en lugares donde se espera una `Persona`. Esto hace que la herencia sea inútil.
- **Normalización:** Al trabajar con herencia y con muchos campos las tablas de la base de datos pierden las formas normales. Esto conlleva a mal rendimiento y uso de la base de datos.
- **Lógica duplicada:** Cuando la abstracción y el uso concreto no coinciden es necesario duplicar mucha lógica de la clase padre, y recién ahí se puede colocar la lógica de negocio específica. Incluso aunque se puedan sobrescribir métodos, la lógica ya se duplicó.
- **Sistema frágil:** En el ejemplo de `Persona` y `Estudiante` se trató el `Apellido` vs `Apellido paterno/materno`. El problema concreto es que: `Estudiante` aplicó su propia lógica. Ahora, para intentar mantener la `Persona` intacta, es necesario actualizar 2 cosas. Esto es propenso a errores, un miembro del equipo (o una IA) puede olvidar que se actualizan 2 entidades, y así el sistema se rompe

de forma críptica. Este es un problema que sucederá comúnmente en todas las cosas que toquen las librerías.

- **Fuente de la verdad:** Cuando las librerías se ignoran y se crea funcionalidad personalizada (como Estudiante) ya no existe una fuente de la verdad. Ahora hay 2 o más lugares que actualizar y sincronizar.
- **Principios OOP faltantes:** La propuesta existente se escuda demasiado en principios y prácticas, y no observa la realidad de que se están rompiendo estas prácticas. En vez de discutir si las librerías están bien planteadas o bien ejecutadas se desvía la conversación hacia los principios de OOP. Los principios son importantes y traen beneficios, pero ¿qué caso tiene hablar de principio de responsabilidad única, abierto cerrado, inversión de dependencias, liskov, etc., cuando la implementación presentada no los sigue?

Frente a los problemas propuestos la respuesta no puede ser "ten cuidado" o "sigue las buenas prácticas". Existen problemas reales que necesitan soluciones reales. El avance de la computación trae consigo mejores herramientas y técnicas que nos permiten enforzar límites. Ese es el punto del DDD, tener límites claros sobre qué se puede o no hacer. Tener invariantes, precondiciones, postcondiciones, y propiedades en nuestro sistema. El lenguaje (C#) y la plataforma (.NET) nos permiten expresar esas reglas, los compiladores las verifican y validan. La implementación existente abandona varios de estos en nombre de una utopía de sistema genérico, poniendo el carro frente al caballo.

III. TALLER O FÁBRICA

Como un pequeño inciso, se presentó la idea de pasar de ser un taller a ser una fábrica. Dejar de ser artesanos para ser operarios. Sin embargo, la analogía misma resume los problemas importantes mostrados anteriormente.

Asumiendo que vamos a seguir desarrollando software a medida, no podemos ser una fábrica. Porque no vamos a fabricar un único modelo de auto y vendérselo a todos los clientes. Eso sería crear un sistema SaaS, y Odoos studio/sh ya existen.

Empezar a construir ahora una fábrica de Sedans, para que venga un cliente que necesita un 4x4, es un despropósito. Hemos invertido mucho tiempo y dinero en sedans, para deshacer gran parte del trabajo y empezar de nuevo en 4x4.

Bueno, entonces trabajemos no en una fábrica de autos, sino en una fábrica de chasis, de ejes, de llantas. ¿Qué hacemos cuando el cliente no quiere un auto, sino un camión? ¿Una mototaxi? ¿Un auto para funeraria? El mismo problema, hay que deshacer y rehacer.

¿Qué podemos hacer que sea reutilizable? ¿Ejes? ¿Llantas? ¿Lunas? Una fábrica así acabaría fabricando partes individuales pequeñas, o trabajando solo en modelos muy similares.

Intentar crear una única fábrica de sistemas para CRM, Colegio, Inventarios, Clínicas, Manufactura, etc., y seguir trabajando a medida, es una tarea inmensa y ardua, que requiere de bastante trabajo y diálogo.

IV. NUEVA PROPUESTA PARA LA REUTILIZACIÓN

A continuación se presenta una nueva propuesta para la reutilización de código en la empresa. Esta propuesta responde a las preguntas planteadas, ofrece claridad en los términos y gobernanza, y está abierta a mejoras por parte del equipo.

A. Definiciones

Existe mucha confusión y debate entre la terminología, por lo que a continuación se definen las cosas importantes, y cómo es que se utilizarán más adelante.

- **Librería:** Una colección de herramientas (funciones, clases, variables) que facilitan una tarea. Se hace énfasis en **herramienta**. Un conjunto de funciones para interactuar con PostgreSQL es una librería, sin embargo, un `Estudiante` no puede ser una librería, es una entidad y no es muy reutilizable.

En particular, una librería permanece completamente fuera del sistema que la utiliza. Solo se utiliza la librería, no se la modifica para que se adapte a necesidades específicas. Una vez que se modifica, deja de ser una librería y se convierte en parte del sistema.

- **Plantilla:** Una colección de archivos que se utilizan como base para trabajo nuevo. Se copia y pega, y se adapta a las necesidades específicas del problema a resolver.

B. Reusabilidad basada en plantillas

El modelo a utilizar para la reutilización de código es uno basado en plantillas. En lugar de tener una serie de librerías completamente estáticas, se desarrollan plantillas con funcionalidad común. Estas plantillas luego se copian a cada sistema específico. Por ejemplo, en Colver, en lugar de tener una librería `'libs/Common/Person'` que no podemos tocar y que eventualmente generará fricción, simplemente "copiamos" los archivos de `'libs/Common/Person'` hacia nuestro sistema, directo a `'src'`.

Esto tiene los siguientes beneficios:

- La plantilla contiene funcionalidad común y reutilizable, que beneficia directamente al proyecto.
- Nosotros como Colver somos dueños del código. Si necesitamos cambiar algo, porque Colver trabaja de forma distinta, podemos hacerlo sin problemas, y sin tener que duplicar funcionalidad o crear nuevas capas de Anticorrupción.
- Si la plantilla tiene cosas innecesarias o problemáticas para Colver, simplemente las eliminamos, sin temor a romper otros proyectos.
- Las plantillas pueden evolucionar independientemente de Colver.

Durante la presentación anterior se habló de "sacar código" fuera de un sistema específico hacia una librería. Bajo el modelo de plantillas esto es cierto, y es a propósito. Esto se detalla en la sección de construcción y mantenimiento.

La realidad del software a medida es que las necesidades de dos clientes nunca van a ser las mismas. Estas plantillas necesitan alcanzar un balance entre ofrecer suficiente funcionalidad para ser útiles, y no especular demasiado.

C. Caso de éxito: shadcn UI

Shadcn UI es un sistema basado en plantillas que ofrece componentes visuales comunes. Sigue el mismo patrón propuesto aquí: Una serie de archivos iniciales a modo de plantillas, que se copian y pegan en cada proyecto.

Gracias a que cada proyecto es dueño de los componentes es que ha alcanzado la popularidad que posee actualmente, utilizado en una gran cantidad de proyectos en todo el mundo.

D. Gobernanza

Las plantillas se construyen de forma independiente de los proyectos. No se toma código de un proyecto existente y se lo "adapta" o se "quitan" cosas. Esto tiene la ventaja de que se trabaja bajo una base limpia, permitiendo tratar la plantilla de forma realmente genérica.

Las plantillas son desarrolladas por cualquier persona o grupo de personas, y deben ser revisadas y aceptadas por al menos dos tercios del equipo. De este modo se tiene un intercambio de ideas real, cada persona puede sacar a la luz posibles deficiencias derivadas de su experiencia, y se construye una base realmente reutilizable.

Desarrollar una plantilla no requiere ningún documento formal. Solo se requiere el código funcional, un ejemplo de su uso, documentación escrita **manualmente** (no por IA) en archivos markdown o txt, y la aceptación del equipo.

En el caso de que no se llegue a un acuerdo repetidas veces un miembro del equipo deberá tomar la decisión de aceptar la plantilla a pesar de la falta de acuerdos, cerrar su implementación por un plazo de tiempo prudente, u otros que considere adecuados.

Una persona solo puede proponer una plantilla nueva con funcionalidad que haya trabajado personalmente. Es decir, si Juan nunca ha trabajado en Contabilidad, no puede proponer una librería de Contabilidad. Esta medida busca evitar la abstracción especulativa, la sobre-ingeniería, y llenar las plantillas de funcionalidad que no es inmediatamente necesaria.

Las plantillas **deben** incurrir en un uso **moderado a bajo** de inteligencia artificial. La IA es capaz de resolver problemas grandes cuando se les da un contexto y un plan, pero el desarrollo de plantillas es en sí planeamiento.

¿Qué constituye uso moderado a bajo de IA?

- La idea original (Ejm. plantilla de cotizaciones) no puede venir de una IA.
- El flujo de trabajo no puede venir de una IA.
- Las entidades, sus nombres, propiedades y operaciones no pueden venir de una IA.
- La IA puede automatizar la implementación, pero el plan, flujo e ideas no pueden venir de una IA.
- La IA puede dar feedback de un flujo existente, escribir pruebas, código.
- La IA puede escribir docstrings en el código y archivos markdown internos.
- La IA **no puede** escribir archivos de documentación finales. El desarrollador que propone la plantilla **debe** entender todo el código que propone, y debe ser capaz de escribir por sí mismo documentación final.

¿Por qué se limita tanto el uso de IA? Porque se busca generar abstracciones orgánicas, que surjan de los proyectos reales que hemos desarrollado. La IA no tiene el contexto de todos los proyectos pasados, presentes y futuros.

E. Contenido de las plantillas

Cada plantilla consta de:

- Un manifiesto, que detalla sus propiedades y dependencias. Por ejemplo, un archivo json o yaml.
- Una serie de entidades estrictamente relacionadas entre sí, con funcionalidad como métodos.
- Interfaces para repositorios y servicios.
- Una cantidad mínima de casos de uso. Las plantillas deberían buscar encapsular operaciones dentro de las entidades, o en servicios de dominio. Los casos de uso engloban interacción entre varios componentes distintos, y por lo tanto se salen del alcance.
- No debe contener capa de Presentación. Esta capa es para interactuar con usuarios finales del sistema, pero la librería se enfoca en los programadores.
- Si la capa de Infraestructura se puede incluir o no se deja abierto a debate entre los miembros del equipo.

Si se incluyen otros componentes como Value Objects, Agregados, Excepciones, etc., que son de uso común en la arquitectura limpia y DDD, pero que no han sido especificados, se dejan abiertos a debate. En particular, los Value Objects son bastante comunes y se adaptan bien al patrón de Librería, donde se crean una vez y no se vuelven a tocar.

Para usar las plantillas se desarrollará una pequeña herramienta como shadcn que instala la plantilla, y cualquier dependencia que necesite. Luego el equipo podrá utilizarlas como sea necesario.

Las plantillas se desarrollan bajo un modelo de composición en vez de un modelo de herencia. De este modo se mantiene a las librerías útiles por más tiempo. Por ejemplo, en vez de que Estudiante herede de Persona, el estudiante contiene una referencia interna a persona, y delega a este cuando necesita utilizar las operaciones comunes.

Dentro de las plantillas se reduce el polimorfismo al mínimo, y en caso de utilizarse, se trabaja mediante interfaces en vez de clases. Ya que se espera que las plantillas sean modificadas para cada negocio, sería muy común que en estas modificaciones el polimorfismo se rompa. Al depender en interfaces se declara únicamente un comportamiento esperado, en vez de una identidad (is-a vs has-a).

No se tratan las entidades como intercambiables. Es decir, si Estudiante y Profesor componen internamente a una Persona, aun así no se esperará que operaciones de Persona apliquen igualmente a Estudiante y Profesor.

F. Línea de tiempo

La implementación de plantillas es un prospecto a mediano y largo plazo. Se propone que se desarrollen las plantillas tras acabar la implementación de un proyecto, utilizando las lecciones aprendidas en tal proyecto. Esto requiere una inversión inicial de tiempo para la implementación.

Ya se cuenta en cierta medida con una plantilla de Autenticación y Autorización, por lo que al acabar Colver se puede trabajar en la siguiente plantilla. Dependiendo del tiempo y personal disponible, sería posible implementar de una a tres plantillas en cada corte, o mejorar una a dos plantillas en el mismo tiempo.

V. LIMITACIONES

Esta propuesta deja abiertos varios puntos que son necesarios para una implementación exitosa.

- **Normalización:** Al tener la posibilidad de varias entidades compuestas de otras, se siguen perdiendo varias formas normales.
- **Dependencias opcionales:** No se aclaró cómo es que una plantilla (Ejm.: Ventas) interactúa y se configura cuando se incluye una plantilla opcional (Ejm.: Contabilidad).
- **Infraestructura:** Al basarse en un modelo de composición, puede que sea contraproducente que las entidades de una plantilla cuenten con su propia tabla. En vez de eso cada entidad downstream contendría todos los datos necesarios. Sin embargo, este punto necesita más trabajo y diálogo entre el equipo.
- **OOP:** Esta implementación se aleja intencionalmente de la programación orientada a objetos, sin embargo, no están 100% claros qué conceptos aplicar o no.
- **Manejo de errores:** Esta implementación no incluye un marco para el manejo de errores. Esto es particularmente importante ya que C# y ASP.NET no siguen un modelo basado en excepciones, sino que lo dejan abierto a los programadores.
- **Otros aspectos de DDD:** Asimismo, no se especifican detalles acerca de otros puntos como Aggregates, Bounded contexts, Servicios de dominio, Eventos, etc.

VI. RECOMENDACIONES

- Toda la propuesta está abierta a sugerencias y mejoras, ya que lo más importante es el consenso.
- Antes de iniciar la implementación se deberían establecer los puntos faltantes.
- Se recomienda usar un enfoque basado en la simplicidad (principios KISS y YAGNI).

VII. CONCLUSIONES

- Se identificaron los problemas existentes en la propuesta previa, se encontró un uso inadecuado de patrones y OOP, librerías muy rígidas y frágiles, sobre-ingeniería y análisis por parálisis.
- Se identificaron posibles problemas futuros como complejidad, pérdida de agilidad, redundancia y baja alineación.
- Se presentó una propuesta para la reutilización de código basada en composición y plantillas que responde a las críticas anteriores para una mejor reutilización de código.

REFERENCES

- [1] S. Jayashetty, P. K. Manjunatha, and H. Kashyap, "Over-engineering enterprise architecture and business competitiveness," *SETLabs Briefings*, vol. 2, no. 4, 2004.
- [2] S. Wagner and F. Deissenboeck, "Abstractness, specificity, and complexity in software design," in *Proceedings of the 2nd International Workshop on the Role of Abstraction in Software Engineering*, 2008, pp. 35–42.
- [3] N. Badizadegan, "Abstraction is Expensive — specbranch.com," <https://specbranch.com/posts/expensive-abstraction/>, [Accessed 15-03-2026].
- [4] J. Kramer, "Is abstraction the key to computing?" *Communications of the ACM*, vol. 50, no. 4, pp. 36–42, 2007.
- [5] "Composition over inheritance - Wikipedia — en.wikipedia.org," https://en.wikipedia.org/wiki/Composition_over_inheritance, [Accessed 15-03-2026].
- [6] R. Pike, "Less is exponentially more — commandcenter.blogspot.com," <https://commandcenter.blogspot.com/2012/06/less-is-exponentially-more.html>, [Accessed 15-03-2026].
- [7] C. Aesthetics, "Abstraction can make your code worse," <https://www.youtube.com/watch?v=rQIMtztAoA>, [Accessed 15-03-2026].
- [8] —, "The flaws of inheritance," <https://www.youtube.com/watch?v=hxGOiiR9ZKg>, [Accessed 15-03-2026].
- [9] C. Murotori, "The big oops: Anatomy of a thirty-five-year mistake," <https://www.youtube.com/watch?v=wo84LFzx5nI>, [Accessed 15-03-2026].
- [10] —, "Clean code, horrible performance," <https://www.youtube.com/watch?v=tD5NrevFtbU>, [Accessed 15-03-2026].
- [11] B. Will, "Oop is bad," <https://www.youtube.com/watch?v=QM1iUe6IofM>, [Accessed 15-03-2026].